

DVSA Vulnerability Discovery and Remediation

ICS-344 Information Security Course Project

King Fahd University of Petroleum and Minerals (KFUPM)

Term 252

Project Context

This report is the written submission for the ICS-344 Information Security course project at King Fahd University of Petroleum and Minerals (KFUPM). The project description requires students to deploy OWASP DVSA in a controlled non-production AWS environment, investigate the 10 official vulnerability lessons, document reproduction evidence, apply or describe fixes, and verify post-fix behavior.

The DVSA environment used for this submission is a serverless AWS application in `us-east-1`. The main services involved are Amazon S3, API Gateway, AWS Lambda, DynamoDB, SQS, SES, IAM, CloudWatch, and CloudTrail. All sensitive values in the repository are redacted, including JWTs, AWS account IDs, credentials, email addresses, and private URLs.

Deliverables Check

Required deliverable	Repository location			Status
Written report PDF	<code>report/final-report.pdf</code> ; <code>report/main.tex</code>	source	in	Present
Presentation slides	<code>presentation/slides.pdf</code> ; <code>presentation/project-slides.tex</code>	source	in	Present
GitHub repository evidence	<code>vulnerabilities/</code> , <code>fixes/</code> , <code>screenshots/</code> , <code>diagrams/</code> , <code>scripts/</code>			Present
Demo video recording	Google Drive link in <code>videos/README.md</code>			Linked

Repository Validation

Each official lesson folder contains a README with the required 10-part structure, a redacted request file, fix artifacts, and screenshot evidence. Source reports are included for lessons where the team produced separate DOCX or PDF material.

No.	Lesson	README status	sta-	Screenshot evidence	evi-	Fix files
1	Event Injection	10 parts present		6 files		5 files
2	Broken Authentication	10 parts present		8 extracted re- port images		3 files
3	Sensitive Data Exposure	10 parts present		10 files		5 files
4	Insecure Cloud Configu- ration	10 parts present		12 extracted re- port images		3 files
5	Broken Access Control	10 parts present		3 extracted re- port images		3 files
6	Denial of Service	10 parts present		8 named screen- shots		3 files
7	Over-Privileged Function	10 parts present		10 named screenshots		3 files
8	Logic Vulnerabilities	10 parts present		13 extracted re- port images		3 files
9	Vulnerable Dependencies	10 parts present		6 files		5 files
10	Unhandled Exceptions	10 parts present		2 extracted re- port images		3 files

Lesson 1: Event Injection

Part 1: Goal and Vulnerability Summary

The goal is to show that the DVSA order API can mishandle attacker-controlled event data. The affected path is API Gateway to the order-processing Lambda. A crafted request can make data behave like executable backend logic.

Part 2: Root Cause

The root cause is unsafe deserialization or dynamic handling of request-controlled input. The backend trusted structured request data without ensuring that it remained plain JSON data.

Part 3: Environment and Setup

The test uses the DVSA order endpoint in `us-east-1`, the order Lambda workflow, CloudWatch logs, and redacted payloads in `vulnerabilities/lesson-01-event-injection/redacted-requests.txt`.

Part 4: Reproduction Steps

The reproduction locates the API Gateway endpoint, submits a crafted event-injection payload, observes the client response, and checks CloudWatch for backend execution evidence.

Part 5: Evidence and Proof

Evidence is stored in `screenshots/lesson-01/`, including endpoint discovery, the code injection request, normal server response, exploit evidence, and post-fix request evidence. The source evidence PDF is stored as `vulnerabilities/lesson-01-event-injection/report-source.pdf`.

Part 6: Fix Strategy

The fix is to remove unsafe deserialization, parse input with safe JSON parsing, and validate request fields with an allowlisted schema.

Part 7: Code / Config Changes

Before and after examples are stored in `fixes/lesson-01/`. The after state treats the payload as data and rejects unsupported fields.

Part 8: Verification After Fix

The same crafted payload is repeated. The fixed backend rejects it as invalid input and CloudWatch no longer shows injected code execution.

Part 9: Structured Analysis

Intended rule	Deviation	Fix verification
Request data must never execute inside Lambda.	Attacker-controlled serialized content reached backend execution.	Payload rejected without backend execution evidence.

Part 10: Takeaway

Serverless input parsing must treat request bodies as data only. Unsafe parsing can turn an API request into backend code execution.

Lesson 2: Broken Authentication

Part 1: Goal and Vulnerability Summary

The goal is to demonstrate broken JWT authentication in the DVSA order workflow. A forged or modified token can be used to impersonate another user if the backend trusts decoded claims without complete verification.

Part 2: Root Cause

JWT claims are client-visible and can be edited. The weakness occurs when the backend uses identity fields such as username or subject without verifying signature, issuer, audience, expiry, and trusted subject.

Part 3: Environment and Setup

The test uses two DVSA lab users, browser DevTools, the order API, JWT payload inspection, and the imported Lesson 2 source report in `vulnerabilities/lesson-02-broken-authentication/report-source.docx`.

Part 4: Reproduction Steps

Create orders for two users, capture a valid user token, modify the identity claim, send the forged request to the order API, and observe unauthorized access to another user's order data.

Part 5: Evidence and Proof

Evidence is stored in `screenshots/lesson-02/from-report/` with 8 extracted images from the source report. The sanitized request pattern is stored in `redacted-requests.txt`.

Part 6: Fix Strategy

The backend must verify token integrity and trusted claims before using identity. Invalid signatures, wrong issuer or audience, expired tokens, and inconsistent subject claims must be rejected.

Part 7: Code / Config Changes

Before and after authentication examples are stored in `fixes/lesson-02/`. The fix moves trust from decoded payload fields to verified backend identity.

Part 8: Verification After Fix

The forged-token request is repeated. It is rejected, while a valid user token still returns only that user's own orders.

Part 9: Structured Analysis

Intended rule	Deviation	Fix verification
Only verified identity claims may authorize order access.	Caller-controlled claims were treated as proof of identity.	Forged token rejected; normal token still works.

Part 10: Takeaway

Decoding a JWT is not authentication. The backend must verify the token before trusting any claim.

Lesson 3: Sensitive Data Exposure

Part 1: Goal and Vulnerability Summary

The goal is to show that DVSA receipt or storage workflows can expose private data through API responses, generated files, logs, or object access.

Part 2: Root Cause

The root cause is weak authorization and data minimization around receipt-related data. Receipt content or links were exposed without sufficiently proving ownership.

Part 3: Environment and Setup

The test uses API Gateway, Lambda, DynamoDB, S3 receipt objects, CloudWatch, and the Lesson 3 source report at [vulnerabilities/lesson-03-sensitive-data-exposure/report-source.pdf](#).

Part 4: Reproduction Steps

Create normal receipt/order data, trigger the vulnerable receipt path, capture the exposed response or object evidence, and redact identifiers before storing proof.

Part 5: Evidence and Proof

Evidence is stored in `screenshots/lesson-03/`, including account creation, token capture, API endpoint discovery, DynamoDB order evidence, sensitive file evidence, malicious request evidence, and post-fix results.

Part 6: Fix Strategy

Add ownership checks before returning receipt data or signed URLs. Keep receipt objects private, return only necessary fields, and issue short-lived access only after authorization.

Part 7: Code / Config Changes

Before and after examples are stored in `fixes/lesson-03/`. The after state restricts access to authorized owners and removes unnecessary sensitive output.

Part 8: Verification After Fix

Repeat the unauthorized receipt access attempt. The fixed system denies or sanitizes the response while legitimate owner access still works.

Part 9: Structured Analysis

Intended rule	Deviation	Fix verification
Users may access only their own receipt data.	Private data or links crossed the ownership boundary.	Unauthorized access denied or sanitized.

Part 10: Takeaway

Sensitive data controls must be enforced where data leaves the backend, not only where it is stored.

Lesson 4: Insecure Cloud Configuration

Part 1: Goal and Vulnerability Summary

The goal is to demonstrate that insecure S3 configuration can allow untrusted files into the DVSA receipt-processing workflow.

Part 2: Root Cause

The root cause is permissive S3 access combined with Lambda processing that trusts bucket contents. If an unauthorized principal can write receipt-like objects, backend processing may treat them as legitimate.

Part 3: Environment and Setup

The test uses the receipts S3 bucket, the receipt-processing Lambda path, AWS Console or CLI checks, and the imported source report in `vulnerabilities/lesson-04-insecure-cloud-configuration/report-source.docx`.

Part 4: Reproduction Steps

Inspect S3 permissions, attempt a controlled upload of a receipt-like object, observe backend processing, and capture S3 and Lambda evidence.

Part 5: Evidence and Proof

Evidence is stored in `screenshots/lesson-04/from-report/` with 12 extracted screenshots. The sanitized CLI pattern is stored in `redacted-requests.txt`.

Part 6: Fix Strategy

Enable S3 Block Public Access, restrict bucket policy and ACLs, validate object keys and file types, and allow only trusted application roles to write objects that trigger Lambda processing.

Part 7: Code / Config Changes

Configuration notes are stored in `fixes/lesson-04/`. The documented fix enables public access blocking and restricts unintended uploads.

Part 8: Verification After Fix

Repeat the unauthorized upload or public access attempt. It returns access denied, while the legitimate receipt workflow remains functional.

Part 9: Structured Analysis

Intended rule	Deviation	Fix verification
Only trusted roles may write receipt objects processed by Lambda.	The bucket accepted input outside the intended trust boundary.	Unauthorized write or public access fails.

Part 10: Takeaway

Cloud configuration is part of application security. A storage misconfiguration can become a backend attack path.

Lesson 5: Broken Access Control

Part 1: Goal and Vulnerability Summary

The goal is to show that a normal DVSA user can reach privileged order update behavior that should be restricted to administrative logic.

Part 2: Root Cause

The backend accepts a valid user context but does not enforce the required role or group before sensitive functionality. IAM invoke permissions may also allow public-facing paths to reach internal functions.

Part 3: Environment and Setup

The test uses the order API, admin update logic, JWT claims, Lambda invoke permissions, and the imported source report in `vulnerabilities/lesson-05-broken-access-control/report-source.docx`.

Part 4: Reproduction Steps

Create an order as a normal user, capture the user token and order ID, send a crafted update request toward the privileged path, and observe unauthorized order state modification.

Part 5: Evidence and Proof

Evidence is stored in `screenshots/lesson-05/from-report/` with 3 extracted screenshots. The sanitized request example is stored in `redacted-requests.txt`.

Part 6: Fix Strategy

Enforce admin role checks server-side before privileged updates. Restrict Lambda invoke permissions so public-facing functions cannot invoke arbitrary internal functions.

Part 7: Code / Config Changes

Remediation notes are stored in `fixes/lesson-05/`. The fix combines backend authorization checks with narrower IAM permissions.

Part 8: Verification After Fix

Repeat the normal-user privileged update attempt. It is denied, while normal order creation and billing still work.

Part 9: Structured Analysis

Intended rule	Deviation	Fix verification
Only authorized admin logic may update privileged order state.	A non-admin reached admin behavior.	Unauthorized update denied; normal workflow preserved.

Part 10: Takeaway

Authentication proves who the caller is. Authorization still has to decide what that caller may do.

Lesson 6: Denial of Service

Part 1: Goal and Vulnerability Summary

The goal is to demonstrate that repeated or concurrent billing requests can reduce availability in the DVSA order workflow. The affected function is `DVSA-ORDER-BILLING`.

Part 2: Root Cause

The billing path performs backend work for each request and did not sufficiently limit repeated or concurrent requests at the API edge or function layer.

Part 3: Environment and Setup

The test uses the `/dvsa/order` billing action, API Gateway, Lambda metrics, CloudWatch logs, PowerShell request tests, and redacted JWTs.

Part 4: Reproduction Steps

Confirm normal checkout, capture the billing request, send a baseline request, run a controlled repeated request test, run a controlled 10-job parallel test, and compare HTTP results and backend metrics.

Part 5: Evidence and Proof

Evidence is stored in `screenshots/lesson-06/`. Screenshots 6.1 through 6.8 show normal workflow, captured request details, repeated request output, Lambda metrics, CloudWatch logs, and API Gateway throttling.

Part 6: Fix Strategy

Apply API Gateway throttling and backend idempotency or rate-control checks so duplicate billing attempts are rejected cheaply and consistently.

Part 7: Code / Config Changes

Before, after, and explanation files are stored in `fixes/lesson-06/`. The documented control is throttling plus billing idempotency.

Part 8: Verification After Fix

Repeat the controlled test. Excess requests should be throttled or rejected cleanly while normal checkout still succeeds.

Part 9: Structured Analysis

Intended rule	Deviation	Fix verification
Billing must degrade safely under duplicate or burst requests.	A small parallel burst produced mixed 200, 500, and 502 responses.	Throttling evidence and normal checkout confirmation.

Part 10: Takeaway

Availability is a security property. Critical serverless workflows need throttling, idempotency, and graceful failure.

Lesson 7: Over-Privileged Function

Part 1: Goal and Vulnerability Summary

The goal is to demonstrate that the `DVSA-SEND-RECEIPT-EMAIL` Lambda execution role had permissions broader than the receipt-email workflow required.

Part 2: Root Cause

Lambda code runs with execution-role credentials. If a role allows broad access, any abused function path can use those unrelated permissions, increasing blast radius.

Part 3: Environment and Setup

The test uses Lambda, IAM, SES, S3, DynamoDB, CloudWatch, IAM Policy Simulator, and redacted role/policy examples.

Part 4: Reproduction Steps

Open the receipt Lambda, inspect its execution role and attached policies, simulate SES/S3/DynamoDB actions, replace broad permissions with least privilege, rerun simulator checks, and confirm normal order workflow.

Part 5: Evidence and Proof

Evidence is stored in `screenshots/lesson-07/`. It includes Lambda page, execution role, broad SES policy, simulator checks, post-fix simulation, and normal workflow screenshots. The blast-radius diagram is in `diagrams/lesson-07-blast-radius.png`.

Part 6: Fix Strategy

Remove broad managed policies and wildcard resources. Allow only the minimum CloudWatch logging and SES send permissions needed for the receipt workflow.

Part 7: Code / Config Changes

The before and after IAM policy examples are stored in `fixes/lesson-07/before-policy.json` and `fixes/lesson-07/after-policy.json`.

Part 8: Verification After Fix

IAM Policy Simulator shows unrelated actions denied, and normal order/receipt behavior still works.

Part 9: Structured Analysis

Intended rule	Deviation	Fix verification
The receipt function may use only permissions required for receipts and logs.	Broad permissions increased blast radius.	Simulator denial results plus normal workflow evidence.

Part 10: Takeaway

The Lambda execution role is a serverless security boundary. Least privilege limits damage when function code is abused.

Lesson 8: Logic Vulnerabilities

Part 1: Goal and Vulnerability Summary

The goal is to demonstrate a business logic race condition where billing and order update operations can be submitted close together, producing inconsistent order state.

Part 2: Root Cause

The root cause is a non-atomic read/check/write sequence. Billing trusted state that could change before the final write.

Part 3: Environment and Setup

The test uses DVSA-ORDER-BILLING, DVSA-ORDER-UPDATE, DynamoDB order state, and the imported Lesson 8 source report.

Part 4: Reproduction Steps

Create an order, submit billing and update requests close together, inspect charged amount and final order state, and compare expected state with actual state.

Part 5: Evidence and Proof

Evidence is stored in `screenshots/lesson-08/from-report/` with 13 extracted screenshots from `report-source.docx`. A sanitized workflow request is stored in `redacted-requests.txt`.

Part 6: Fix Strategy

Use DynamoDB conditional writes or transactions for order state transitions. Lock or reject updates after billing begins.

Part 7: Code / Config Changes

Before and after state-management examples are stored in `fixes/lesson-08/`. The fix adds a conditional expression to the billing update.

Part 8: Verification After Fix

Repeat the timing test. One operation succeeds cleanly and the conflicting operation is rejected.

Part 9: Structured Analysis

Intended rule	Deviation	Fix verification
Billing and order updates must preserve one consistent order state.	State changed inside a timing window trusted by billing.	Conditional write rejects stale or conflicting update.

Part 10: Takeaway

Valid requests can become unsafe in the wrong sequence. Security-sensitive workflows need atomic state transitions.

Lesson 9: Vulnerable Dependencies

Part 1: Goal and Vulnerability Summary

The goal is to show that an unsafe Node.js dependency can create backend execution risk in the DVSA order request path.

Part 2: Root Cause

The vulnerable package accepts dangerous serialized function markers. Without dependency review, pinning, and scanning, unsafe package behavior becomes part of the application attack surface.

Part 3: Environment and Setup

The test uses the Node.js Lambda package, order request processing, the Lesson 1/9 source report, and CloudWatch evidence.

Part 4: Reproduction Steps

Identify the vulnerable dependency, relate package behavior to event injection, demonstrate the behavior in the controlled lab, and document the remediation path.

Part 5: Evidence and Proof

Evidence is stored in `screenshots/lesson-09/`, including endpoint discovery, injection request, normal response, exploit evidence, and post-fix request evidence. The shared source PDF is stored as `vulnerabilities/lesson-09-vulnerable-dependencies/report-source.pdf`.

Part 6: Fix Strategy

Remove or replace unsafe packages, use safe JSON parsing, pin reviewed versions, and add dependency scanning to the build and deployment process.

Part 7: Code / Config Changes

Before and after dependency remediation examples are stored in `fixes/lesson-09/`. The fix removes unsafe deserialization from request parsing.

Part 8: Verification After Fix

Repeat the injection payload and dependency scan. The payload no longer executes and the risky dependency is absent or remediated.

Part 9: Structured Analysis

Intended rule	Deviation	Fix verification
Dependencies must not evaluate attacker-controlled request data.	A third-party package enabled executable deserialization.	Payload rejected and dependency removed or replaced.

Part 10: Takeaway

Dependencies are part of the attack surface. Secure code can still fail if unsafe packages are deployed with it.

Lesson 10: Unhandled Exceptions

Part 1: Goal and Vulnerability Summary

The goal is to demonstrate unsafe exception handling in the DVSA order API. Malformed input can trigger backend exceptions that expose internal details to clients.

Part 2: Root Cause

The backend assumes required fields exist and does not consistently catch exceptions at the handler boundary. This can leak error names, file paths, line numbers, or stack details.

Part 3: Environment and Setup

The test uses the `/dvsa/order` endpoint, API Gateway, the affected Lambda handler, CloudWatch, and the imported Lesson 10 source report.

Part 4: Reproduction Steps

Capture a normal order request, remove or alter required fields, send the malformed request, observe the error response, and redact any sensitive details.

Part 5: Evidence and Proof

Evidence is stored in `screenshots/lesson-10/from-report/` with 2 extracted screenshots. The sanitized malformed request shape is stored in `redacted-requests.txt`.

Part 6: Fix Strategy

Validate input before business logic and add centralized error handling. Client responses should be generic, while detailed diagnostics remain in CloudWatch.

Part 7: Code / Config Changes

Before and after safe error handling notes are stored in `fixes/lesson-10/`.

Part 8: Verification After Fix

Repeat the malformed request. The client receives a controlled safe error and backend details remain only in logs.

Part 9: Structured Analysis

Intended rule	Deviation	Fix verification
Client errors must not expose backend internals.	Diagnostic details were returned to the caller.	Generic safe error returned; detailed log retained internally.

Part 10: Takeaway

Errors are part of the public interface. Backend diagnostics belong in logs, not in client responses.

Final Redaction and Consistency Notes

The text artifacts were checked for common secret patterns. The only AWS secret-like string found in text is the intentional redacted example `AWS_SECRET_ACCESS_KEY=REDACTED` in the repository README. The submitted evidence should still be visually reviewed before final upload because screenshots can contain values that text scanning cannot inspect reliably.